

previous lecture review: fork the course repo and submit a PR

Fork repo → clone the repo using your terminal

previous lecture review: fork the course repo and submit a PR

Fork repo → clone the repo using your terminal

```
git remote add upstream https://github.com/changhoonhahn/ast384c.git
```

*Your fork already has a remote called origin (your copy on GitHub). This adds a second remote named **upstream**, pointing to the original repo you forked from. Without this, git has no idea the original repo exists.*

previous lecture review: fork the course repo and submit a PR

Fork repo → clone the repo using your terminal

```
git remote add upstream https://github.com/changhoonhahn/ast384c.git
```

```
git remote -v      # verify (-v = verbose)
```


previous lecture review: fork the course repo and submit a PR

Fork repo → clone the repo using your terminal

```
git remote add upstream https://github.com/changhoonhahn/ast384c.git
```

```
git remote -v      # verify (-v = verbose)
```

git fetch upstream *# downloads all commits/branches from upstream into local repo*

git checkout main *# switches your working directory to your local main branch*

git merge upstream/main *# take the commits from upstream/main and merges them into main*

git push origin main *# uploads your now-updated local main to your fork on GitHub (origin)*

some scientific computing basics

python, conda, ssh, HPC

August 26, 2026

PSA: stop scripting/notebooking — start a project

a 400 line `analysis_v2.py` that only runs on your laptop is a script, not a project

a 500 cell `analysis_v3.ipynb` notebook that only runs on your laptop — hidden states, not testable, difficult to code review, difficult to reuse or distribute

a real project separates reusable code from one-off exploration and can be installed, tested, and used by collaborators

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

anatomy of a project

my_project/

```
|— src/
|   └─ my_project/
|       ├── __init__.py
|       └─ models.py
```

```
|— tests/
|   └─ test_models.py
|— pyproject.toml
|— environment.yml
|— README.md
└─ .gitignore
```

code meant to be reused

notes and scripts import from it

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

the importable package



anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

marks this as a package

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

← reusable functions and classes

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

test suite

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

← package metadata, dependencies

anatomy of a project

```
my_project/  
├── src/  
│   ├── my_project/  
│   │   ├── __init__.py  
│   │   └── models.py  
├── tests/  
│   └── test_models.py  
├── pyproject.toml  
├── environment.yml  
├── README.md  
└── .gitignore
```

conda environment spec

anatomy of a project

```
my_project/
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   └── models.py
│   └── tests/
│       └── test_models.py
├── pyproject.toml
├── environment.yml
├── README.md
└── .gitignore
```

mark down explaining project and
how to run it



anatomy of a project

```
my_project/
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   └── models.py
│   └── tests/
│       └── test_models.py
├── pyproject.toml
├── environment.yml
├── README.md
└── .gitignore
```

← files git should never track

pyproject.toml: the project's ID card

```
[project]
name = "my_project"
version = "0.1.0"
requires-python = ">=3.11"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

Declares what your project is and what it needs. Once it exists,
`pip install -e .` makes your package importable anywhere on your machine.

why isolate environments at all?

say two projects need numpy 1.24 vs. numpy 2.0 — they can't both live in one global Python install

“works on my machine” — almost always missing or mismatched dependencies

an isolated, declared environment makes your results reproducible by you or a collaborator months or years from now

conda for scientific computing

one isolated, self-contained environment per project

```
conda create -n ast384c python=3.11
conda activate ast384c
conda install numpy scipy astropy

# when you're done for the day
conda deactivate
```

everything you `pip/conda install` after `activate` stays inside that environment only

environment.yml: share your setup

```
name: ast384c
channels:
  - conda-forge
dependencies:
  - python=3.11
  - numpy
  - scipy
  - astropy
```

commit this to the repo then anyone can then run `conda env create -f environment.yml` and get the same conda environment

conda tips

never install anything into (base) — always activate a project environment first

`conda env list` — to see what you have

`conda env remove -n <name>` to clean up old ones

in-class activities

install conda (I could recommend Miniconda)

set up a conda environment for the course and install `astroropy`

setup a project with `pyproject.toml` + `src/` then `pip install -e .`

help each other

SSH and HPCs

SSH (Secure Shell): an encrypted remote terminal connection

```
ssh yourusername@ls6.tacc.utexas.edu
```

`~/.ssh/config` — no need to retype hostnames

```
Host stampede3  
    HostName stampede3.tacc.utexas.edu  
    User yourusername
```

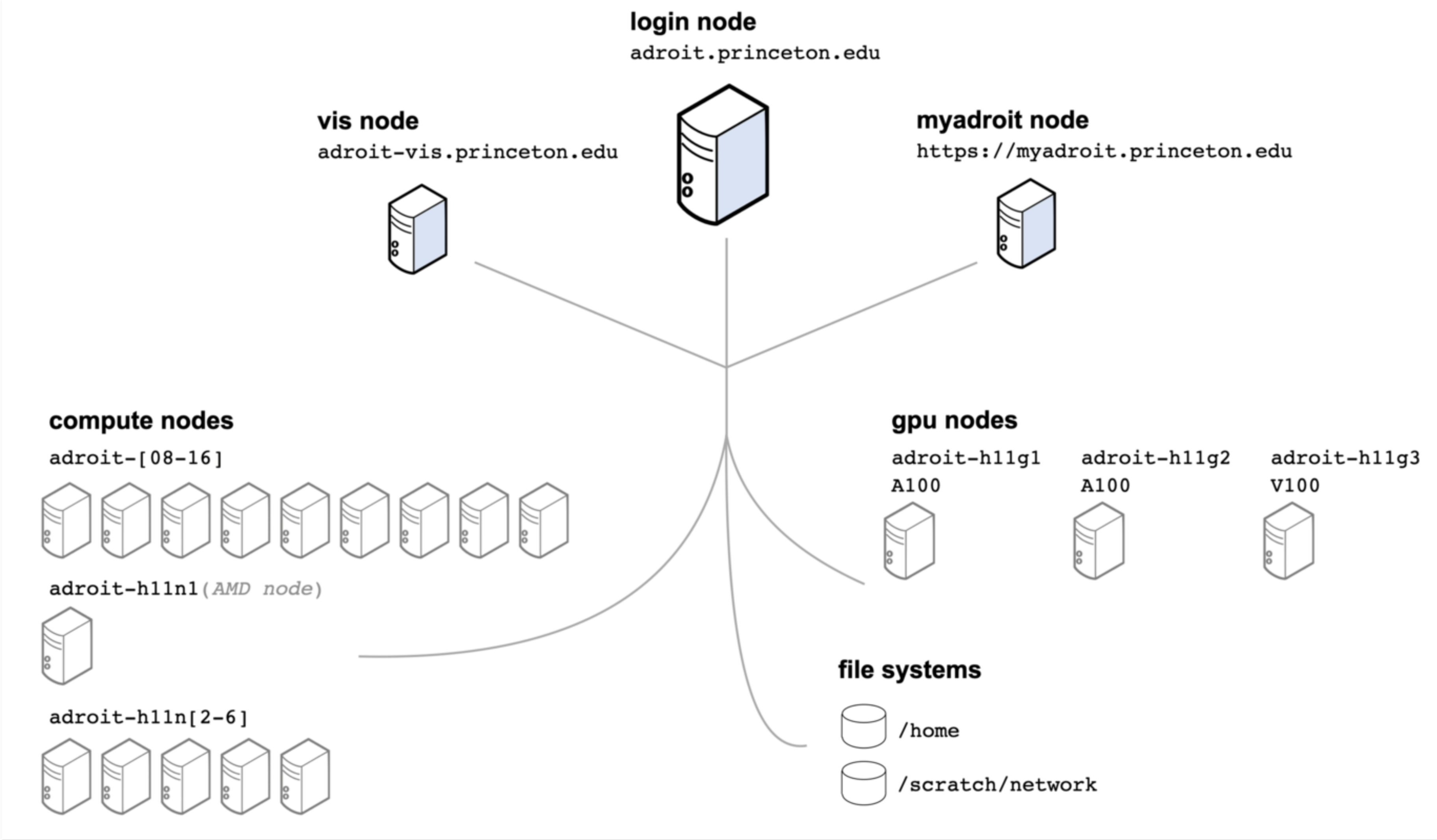

tmux: survive a dropped connection!

if you close your laptop or wifi, your SSH session dies, and anything running in it dies too

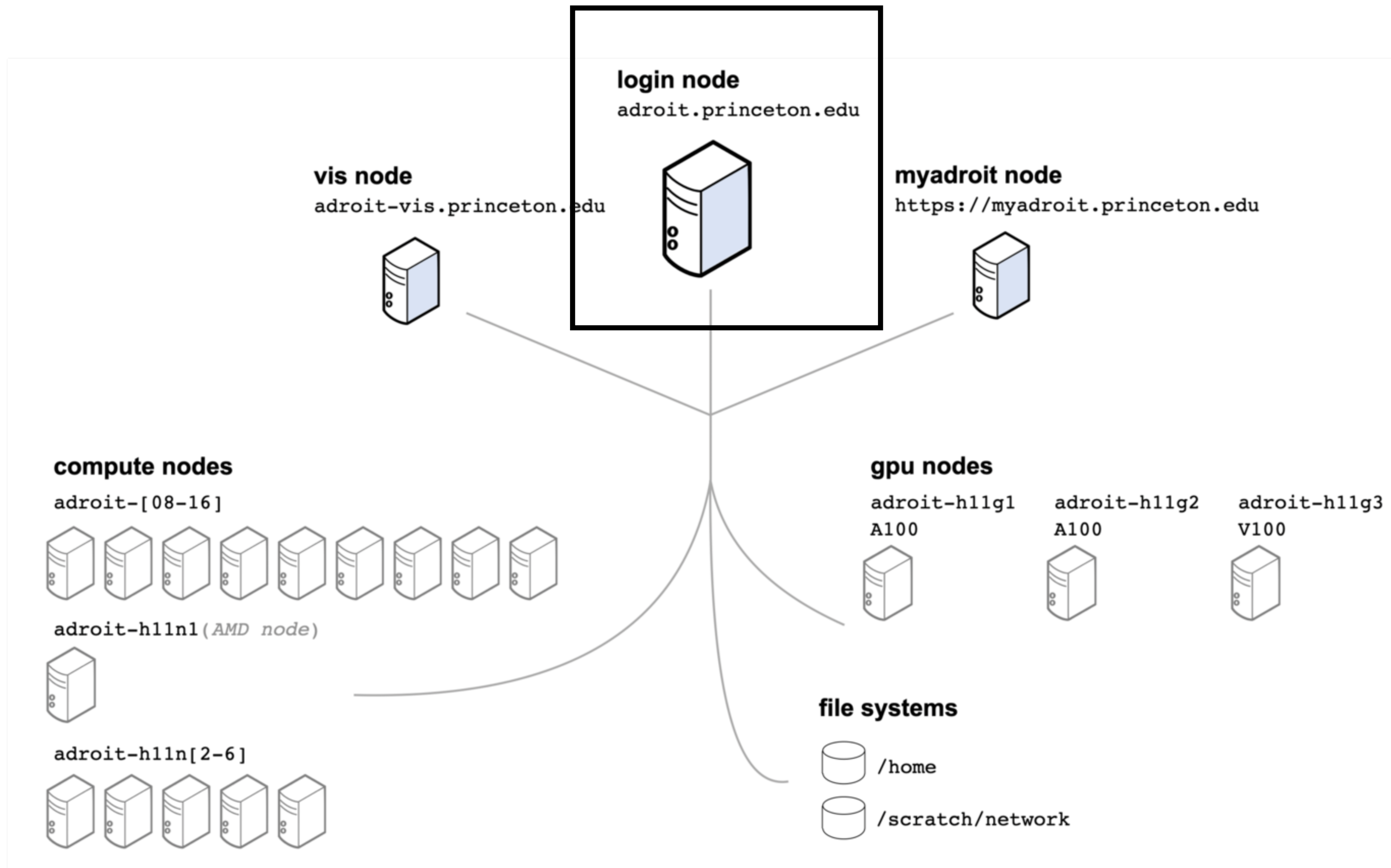
tmux is a terminal session that keeps running even after you disconnect

```
tmux new -s work          # start a named session  
# ... Ctrl-b then d to detach ...  
tmux attach -t work       # reattach later, from anywhere
```

HPC

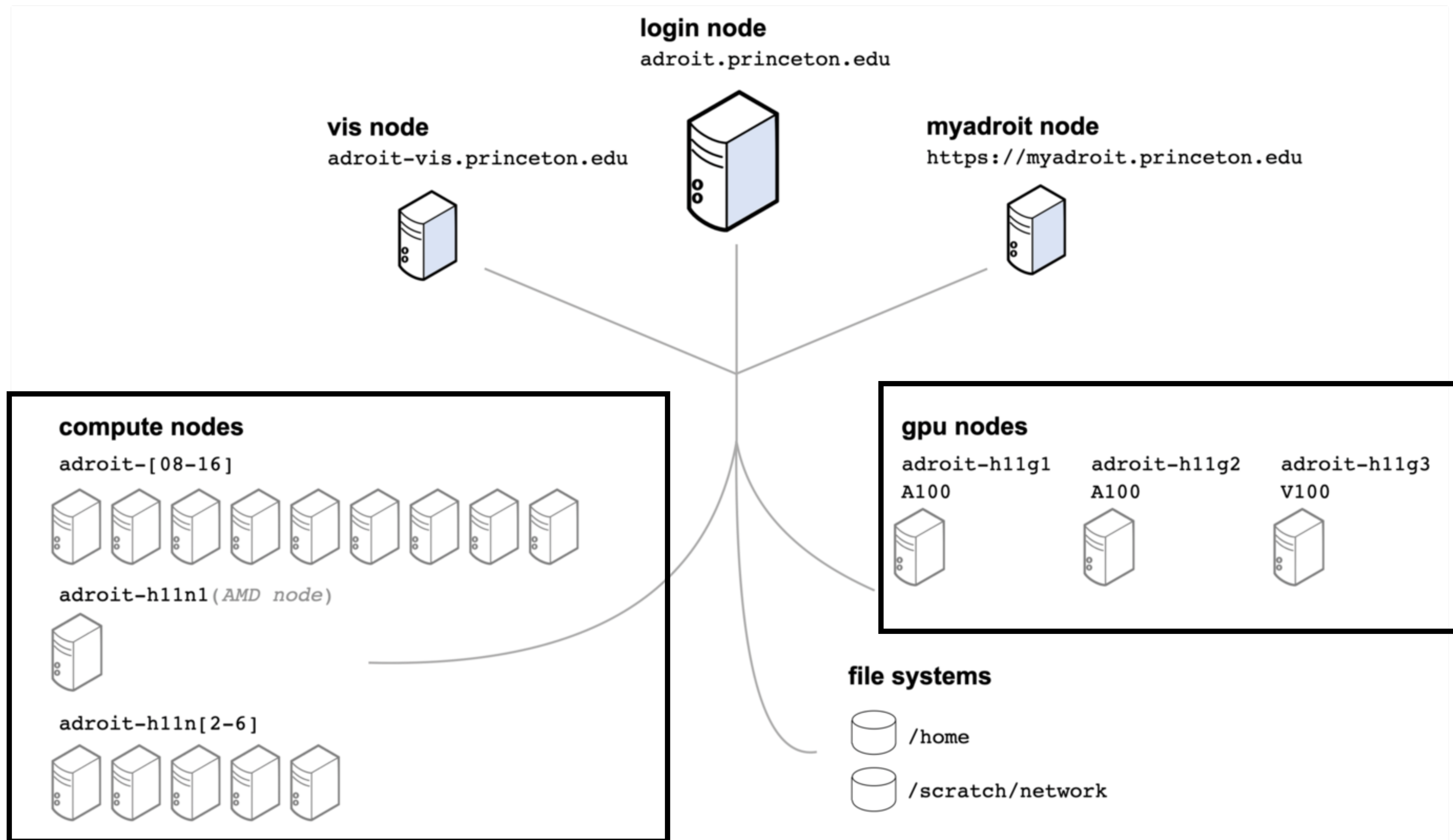


HPC



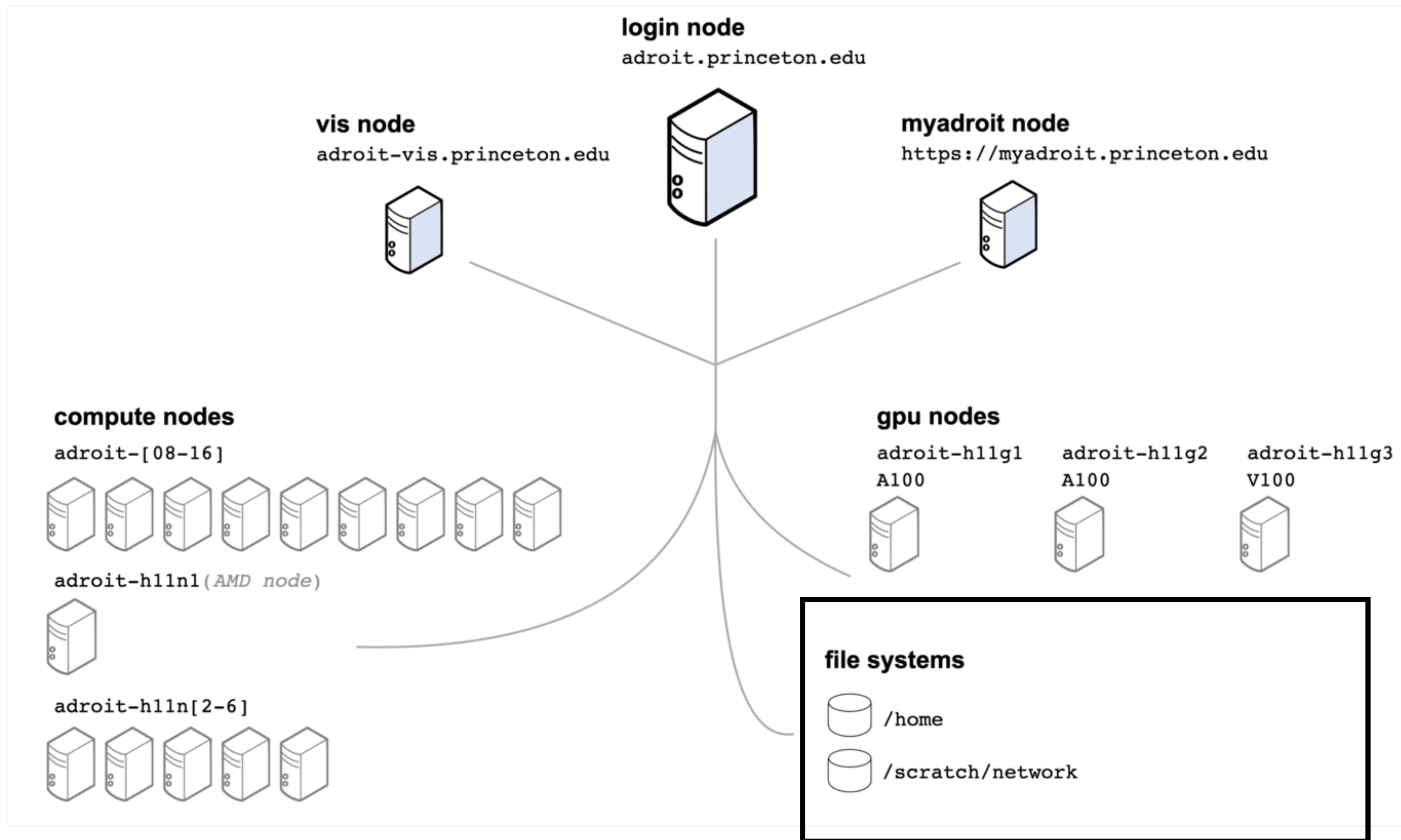
login node: where you ssh into — for editing files and submitting jobs, **not** for running heavy computation

HPC



computing node: where work actually happens, reachable only through the job scheduler

HPC



shared filesystem: `$HOME` versus `$SCRATCH` (gets purged) versus `$WORK` versus long-term/tape storage

environment **modules** on HPC

clusters use **modules** instead of installing everything system-wide. Pair a loaded module with your own conda environment for the Python packages on top.

```
module avail          # what's installed
module load python3/3.11
module list           # what's currently loaded
```

SLURM job scheduler

you submit a job description and SLURM decides when and where it runs

Queue Name	Min/Max Nodes per Job (assoc'd cores)*	Max Job Duration	Max Nodes per User	Max Jobs per User	Max Submit	Charge Rate (per node-hour)
development	8 nodes (1024 cores)	2 hours	8	1	3	1 SU
gpu-a100	8 nodes (1024 cores)	48 hours	12	8	32	3 SUs
gpu-a100-dev	2 nodes (256 cores)	2 hours	2	1	3	3 SUs
gpu-a100-small**	1 node	48 hours	3	3	12	1.5 SUs
gpu-h100	1 node	48 hours	1	1	4	6 SUs
large*	65/256 nodes (65536 cores)	48 hours	256	1	4	1 SU
normal	1/64 nodes (8192 cores)	48 hours	64	20	100	1 SU
vm-small**	1 node (16 cores)	48 hours	4	4	16	0.143 SU

SLURM batch script

```
#!/bin/bash
#-----
# Sample Slurm job script
#   for TACC Lonestar6 AMD Milan nodes
#
#   *** Serial Job in Normal Queue***
#
# Last revised: October 22, 2021
#
# Notes:
#
# -- Copy/edit this script as desired.  Launch by executing
#    "sbatch milan.serial.slurm" on a Lonestar6 login node.
#
# -- Serial codes run on a single node (upper case N = 1).
#    A serial code ignores the value of lower case n,
#    but slurm needs a plausible value to schedule the job.
#
# -- Use TACC's pylauncher utility to run multiple serial
#    executables at the same time, execute "module load pylauncher"
#    followed by "module help pylauncher".
#-----

#SBATCH -J myjob          # Job name
#SBATCH -o myjob.o%j      # Name of stdout output file
#SBATCH -e myjob.e%j      # Name of stderr error file
#SBATCH -p normal         # Queue (partition) name
#SBATCH -N 1              # Total # of nodes (must be 1 for serial)
#SBATCH -n 1              # Total # of mpi tasks (should be 1 for serial)
#SBATCH -t 01:30:00       # Run time (hh:mm:ss)
#SBATCH --mail-type=all   # Send email at begin and end of job
#SBATCH -A myproject      # Project/Allocation name (req'd if you have more than 1)
#SBATCH --mail-user=username@tacc.utexas.edu

# Any other commands must follow all #SBATCH directives...
module list
pwd
date

# Launch serial code...
./myprogram              # Do not use ibrun or any other MPI launcher
```

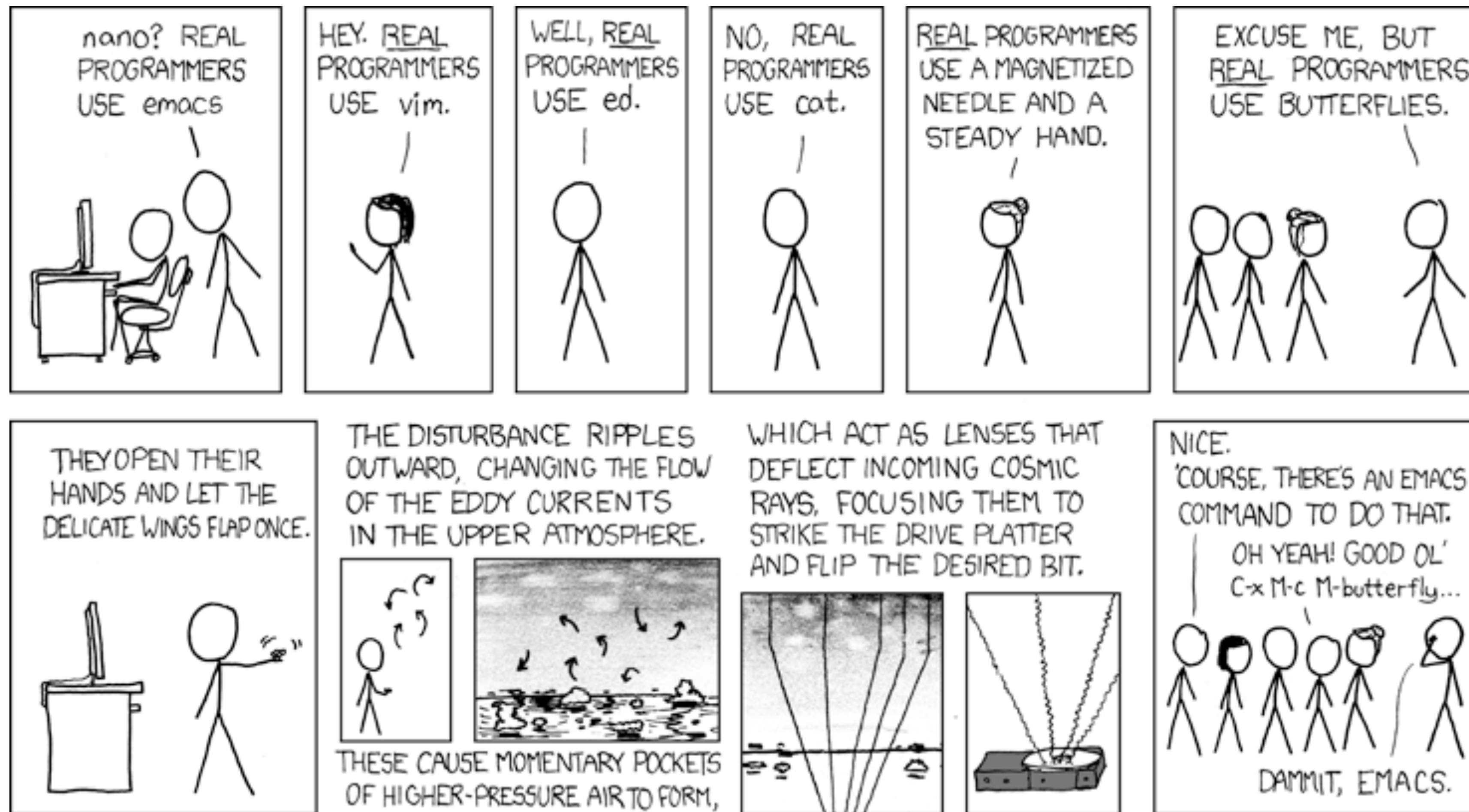

SLURM batch script

```
sbatch job.slurm      # submit  
squeue -u $USER       # check status  
scancel <jobid>       # cancel  
sacct -j <jobid>      # what happened after it finished
```

interactive jobs: for *debugging* and *quick tests* before scaling up to a full batch submission

```
idev -N 1 -n 4 -t 00:30:00
```

vim & friends — lightweight, terminal-based text editor perfect for HPCs



vim & friends — lightweight, terminal-based text editor perfect for HPCs

super-intuitive, shallow learning curve /s

```
vim job.slurm
i           # enter insert mode, type normally
Esc        # back to command mode
:wq        # save and quit
:q!        # quit without saving
```

ships on essentially every HPC so useful to know a few commands

transferring files to and from the cluster

scp — simple copy, fine for one-off single files

rsync — incremental, only copies what changed, resumable; better for large data or whole directories

globus — moving large datasets (many GB to TB+) reliably between HPCs (e.g. *Quijote*, *CAMELS*)

file permissions: chmod

Every file has three permissions (read/write/execute) for three groups of people

-rwxr-xr--			
	read	write	execute
owner (u)	r	w	x
group (g)	r	—	x
others (o)	r	—	—

`chmod u+x file.dat` — symbolic mode: add execute for the owner

`chmod 600 file.dat` — owner read/write only, everyone else nothing

`chmod 755 file.dat` — owner full access, everyone else can read/execute

`chmod 777 file.dat` — everyone has full access

now go break a job script (*on the login node, gently*)

SSH to lonestar6 on TACC (setup a TACC account, setup `~/ .ssh/config`)

<https://aiml-environments-at-tacc.readthedocs.io/en/latest/index.html> installing conda on TACC

submit a small batch job using `inclass/hello.py`, monitor it with `queue`

`rsync` the output files to your machine

references

<https://nbviewer.org/github/jakevdp/WhirlwindTourOfPython/tree/master/>